Lucas Summers
Dr. Alex Dekhtyar
CSC 491
17 March 2026

Senior Project Report

The Problem:

Computer and Data Science students in DATA 301 or CSC 466 classes produce a lot of machine learning models using the popular Python library *scikit-learn*. With these models come many standards metrics and visualizations that help evaluate the model's performance on the data. The issue is that students have many different ways to generate and display these: through scikit-learn built-in functions, custom functions, using other libraries like *Matplotlib*, and more. This can make it hard for professors and graders to see if the student met the requirement of the assignments, as each student may present different deliverables. Along with this, the creation of plots and metrics can often distract from the real intention of the assignments, which is to have students effectively preprocess data and train machine learning models.

The Solution:

We propose a Python package to automatically generate useful evaluation reports for scikit-learn models. The goal is to make it as easy as possible for students to use, simply just plugging in their trained model and the data they used, receiving back a report that they can turn in along with their code. We also wanted to have the report be available to view in many forms, including as an HTML, PDF, JSON and Markdown file (see appendix below for example PDF reports). Last, we wanted to make it available on the Python Package Index (PyPI), such that students working in a Jupyter Notebook can just run '*pip install* ...' to have immediate access to the package.

The Package:

The package is set up in a standard Python package layout. The file *pyproject.toml* defines the metadata necessary for discovery and publishing on PyPI, and it also configures the *setuptools* build system so running '*pip install* ...' includes all required dependencies and package data files (including templates). The implementation code lives under *src/mlreport/*, following current packaging import-path conventions.

Within *src/mlreport/handlers/*, the design is intentionally extensible. The *ModelHandler* class serves as a base class that uses method discovery to identify all available metrics and visualizations for a model type (regression, classification, and clustering). Any subclass can expose a new metric by defining a function with the signature pattern *metric_<id>()*, and can expose a new plot with *plot_<id>()*. The handler reads each method's docstring as the human-readable display name, which is why calling *.available_metrics()* and *.available_plots()* on a report prints both stable IDs and descriptive names.

The *Report* class in *src/mlreport/report.py* acts as the orchestration layer. Along with passing in their trained model, the user is first able to specify some basic metadata, such as the report title, author, description, as well as color themes. Then, it detects whether the model is classification, regression, or clustering and binds the corresponding handler automatically. The workflow is designed to be explicit and chainable, designed to mimic scikit-learn's familiar .fit() and .predict() paradigm. The user-facing API is designed to be simple while still keeping data ingestion, analysis, and rendering separated. The methods are as follows:

- **available_metrics():** Prints all metric IDs supported by the model handler, using function docstrings as readable names.
- **available_plots():** Prints all plot IDs supported by the model handler, using function docstrings as readable names.
- **add_split(name, X, y, y_pred=None):** Adds a named split (e.g., train/test). If *y_pred* is not provided, predictions are generated with *model.predict(X)*. Also, cannot be used after *add_crossval()*.
- **add_crossval(X, y, y_pred=None, cv=None):** Adds cross-validation predictions as a single "cv" split. The CV splitter object used must be passed in the get per-fold statistics in the report. If *y_pred* is not provided, predictions are generated with *cross_val_predict(model, X, y, cv=splitter)*. Also, cannot be used after *add_split()*.
- **add_search(search_cv):** Takes a fitted search CV object (GridSearchCV, RandomizedSearchCV, etc.), and creates a summary of the hyperparameter search with the user's chosen tuning metric. Also generates a top candidates

plot as well as a tuning plot for the report. Currently, only up to two parameters are supported.

- **build(exclude_metrics=None, exclude_plots=None):** Computes metrics and generates plots using the bound handler class. Supports excluding metric or plots IDs by name, which can be viewed when calling *.available_metrics()* or *.available_plots()* on a specific report. This method must be called before *summary()* or any export method.
- **summary():** Prints a concise text summary of the report to the terminal.
- **to_html(path):** Exports full report to HTML using the provided theme template, embedding plot images inline using base64.
- **to_pdf(path):** Exports full report to PDF, using the *weasyprint* library to render the HTML export. This makes the HTML and PDF exports look almost the same.
- **to_md(path, image_dir=None):** Exports report to Markdown and writes plot images to disk (*image_dir* or default *images/* folder)
- **to_json(path):** Exports structured report data to JSON, excluding the plots.

Rendering is template-driven through *Jinja2*. HTML and PDF outputs are generated from theme templates (currently just “light” or “dark”), and Markdown output writes image files and links them in a readable document. We also can set a global *matplotlib* color map, which defines the common color palette used by all the plots. This separation between computed report data and presentation templates makes it easier to update formatting or visual style without changing metric or plotting logic.

At its current stage, classification and regression handlers are implemented with a broad set of useful metrics and plots, while clustering support exists as an extension point for future development. Overall, the package architecture balances ease of use for end users while allowing extensibility for features in the future, such as new model types or even model comparison reports.

Appendix

1. Train/Test Split (Classification)

Classification Report

Lucas Summers
CSC 466-02

MODEL OVERVIEW

Name	LogisticRegression
Type	Classification
Sklearn	1.8.0

HYPERPARAMETERS

C	1.0	class_weight	None
dual	False	fit_intercept	True
intercept_scaling	1	l1_ratio	0.0
max_iter	100	n_jobs	None
penalty	deprecated	random_state	None
solver	lbfgs	tol	0.0001
verbose	0	warm_start	False

DATASET OVERVIEW

Features	5
Total Observations	1000
CV Folds	None

1. Train/Test Split (Classification)

Train	800 (80.0%)
Test	200 (20.0%)

Class Distribution

Class	Train	Test	Overall %
0	403	97	50.0%
1	397	103	50.0%

METRICS (TRAIN/TEST SPLIT)

Metric	Train	Test	Gap
Accuracy	0.8538	0.8800	-0.0262
F1 Score (Macro)	0.8537	0.8800	-0.0262
F1 Score (Weighted)	0.8537	0.8800	-0.0262
Precision (Macro)	0.8537	0.8817	-0.0280
Precision (Weighted)	0.8537	0.8827	-0.0290
Recall (Macro)	0.8537	0.8811	-0.0274
Recall (Weighted)	0.8538	0.8800	-0.0262

*Gap = Train - Test. Large positive values may indicate overfitting.

METRICS (PER-CLASS)

Train

Class	Precision	Recall	F1
0	0.8540	0.8561	0.8550

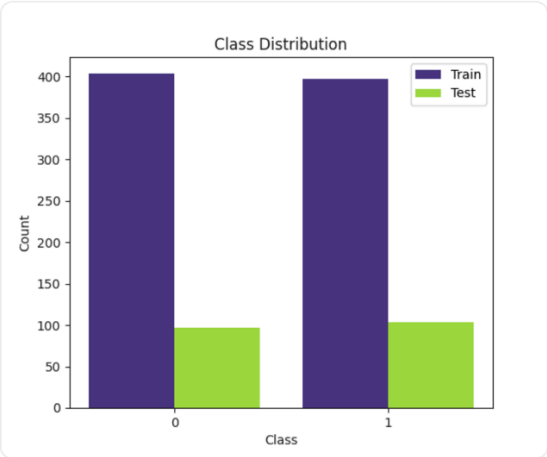
1. Train/Test Split (Classification)

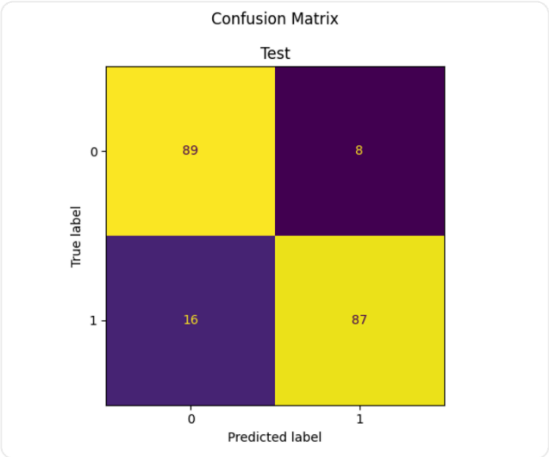
1	0.8535	0.8514	0.8525

Test

Class	Precision	Recall	F1
0	0.8476	0.9175	0.8812
1	0.9158	0.8447	0.8788

VISUALIZATIONS





2. Train/Test Split (Regression)

Regression Report

Lucas Summers
CSC 466-02

MODEL OVERVIEW

Name	`LinearRegression`
Type	`Regression`
Sklearn	`1.8.0`

HYPERPARAMETERS

copy_X	`True`	fit_intercept	`True`
n_jobs	`None`	positive	`False`
tol	`1e-06`		

DATASET OVERVIEW

Features	`5`
Total Observations	`1000`
CV Folds	`None`
Train	`800 (80.0%)`
Test	`200 (20.0%)`

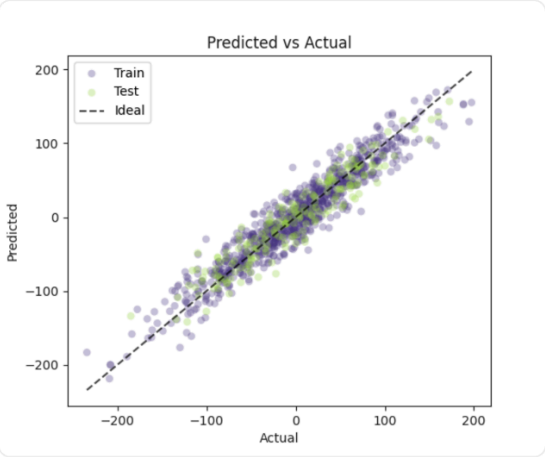
2. Train/Test Split (Regression)

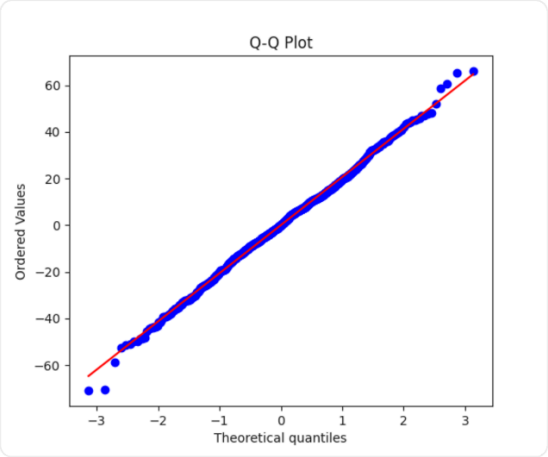
METRICS (TRAIN/TEST SPLIT)

Metric	Train	Test	Gap
Mean Absolute Error	16.2168	17.0702	-0.8534
Max Error	70.6732	61.1649	+9.5083
Median Absolute Error	12.7618	14.8012	-2.0394
Mean Squared Error	424.2005	443.6244	-19.4239
R^2 Score	0.9079	0.8942	+0.0137
Root Mean Squared Error	20.5961	21.0624	-0.4663

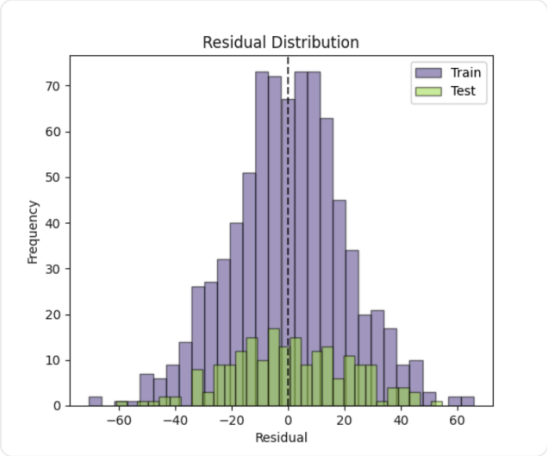
*Gap = Train - Test. Large positive values may indicate overfitting.

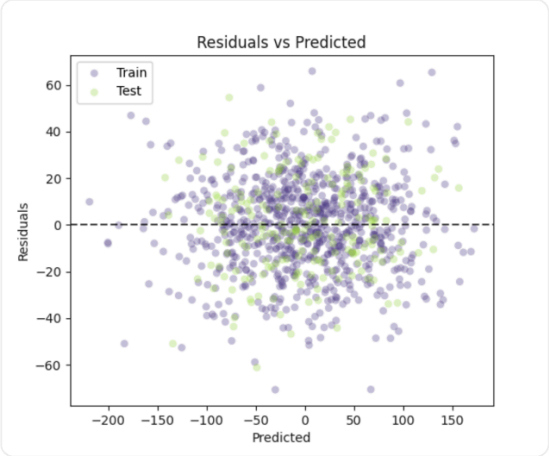
VISUALIZATIONS





2. Train/Test Split (Regression)





3. Cross-Validation Example (Classification)

Classification Report (Cross-Validation)

Lucas Summers
CSC 466-02

MODEL OVERVIEW

Name	LogisticRegression
Type	Classification
Sklearn	1.8.0

HYPERPARAMETERS

C	1.0	class_weight	None
dual	False	fit_intercept	True
intercept_scaling	1	l1_ratio	0.0
max_iter	100	n_jobs	None
penalty	deprecated	random_state	None
solver	lbfgs	tol	0.0001
verbose	0	warm_start	False

DATASET OVERVIEW

Features	5
Total Observations	1000
CV Folds	5

3. Cross-Validation Example (Classification)

Class Distribution

Class	Cv	Overall %
0	500	50.0%
1	500	50.0%

METRICS (CROSS-VALIDATION)

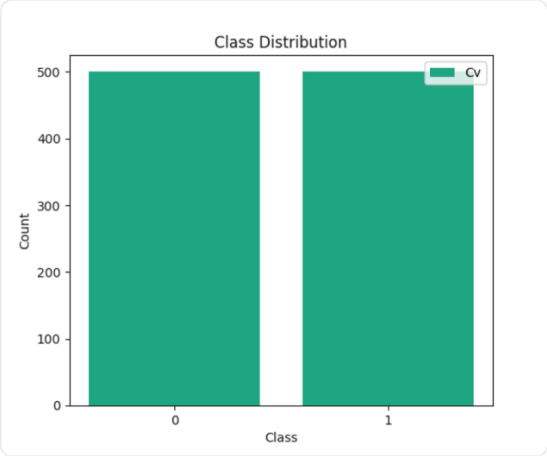
Metric	Mean	Std Dev	Min	Max
Accuracy	0.8600	0.0257	0.8150	0.8850
F1 Score (Macro)	0.8599	0.0258	0.8148	0.8850
F1 Score (Weighted)	0.8599	0.0258	0.8148	0.8850
Precision (Macro)	0.8606	0.0254	0.8166	0.8860
Precision (Weighted)	0.8606	0.0254	0.8166	0.8860
Recall (Macro)	0.8600	0.0257	0.8150	0.8850
Recall (Weighted)	0.8600	0.0257	0.8150	0.8850

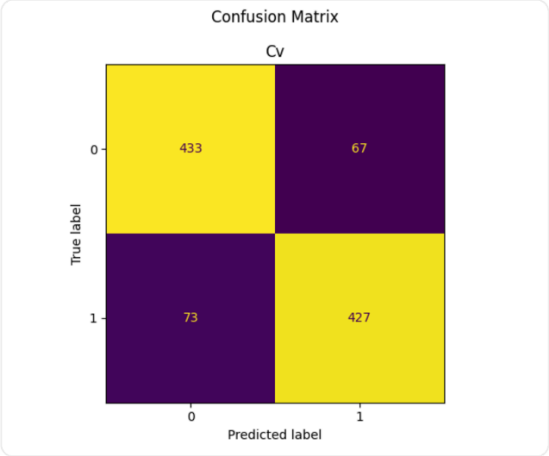
METRICS (PER-CLASS)

Class	Precision	Recall	F1
0	0.8557	0.8660	0.8608
1	0.8644	0.8540	0.8592

3. Cross-Validation Example (Classification)

VISUALIZATIONS





4. Cross-Validation Example (Regression)

Regression Report (Cross-Validation)

Lucas Summers
CSC 466-02

MODEL OVERVIEW

Name	LinearRegression
Type	Regression
Sklearn	1.8.0

HYPERPARAMETERS

copy_X	True	fit_intercept	True
n_jobs	None	positive	False
tol	1e-06		

DATASET OVERVIEW

Features	5
Total Observations	1000
CV Folds	5

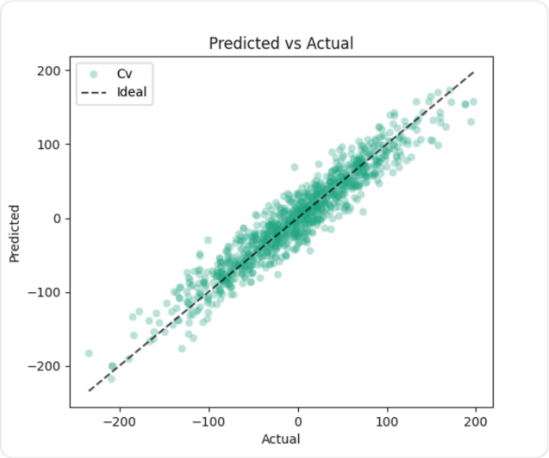
METRICS (CROSS-VALIDATION)

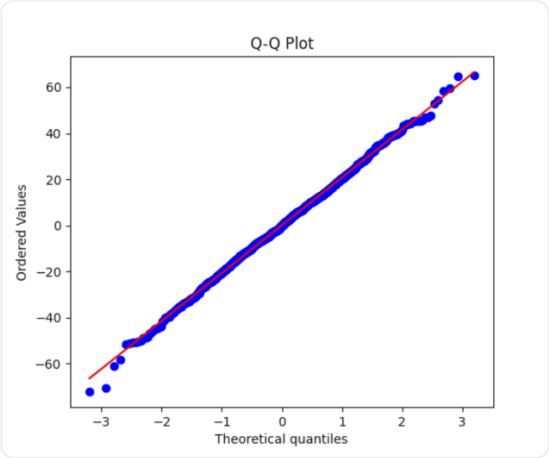
Metric	Mean	Std Dev	Min	Max
Mean Absolute Error	16.4406	0.6681	15.5374	17.2691

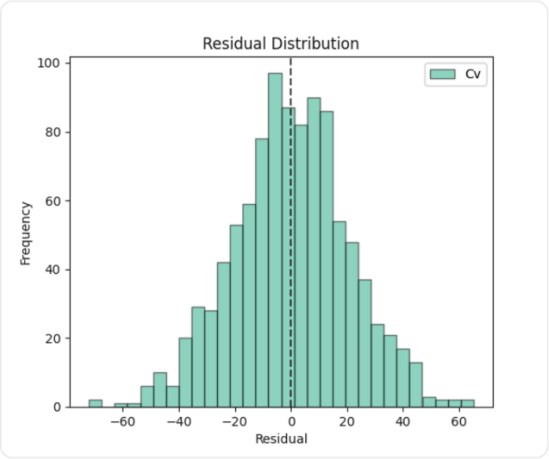
4. Cross-Validation Example (Regression)

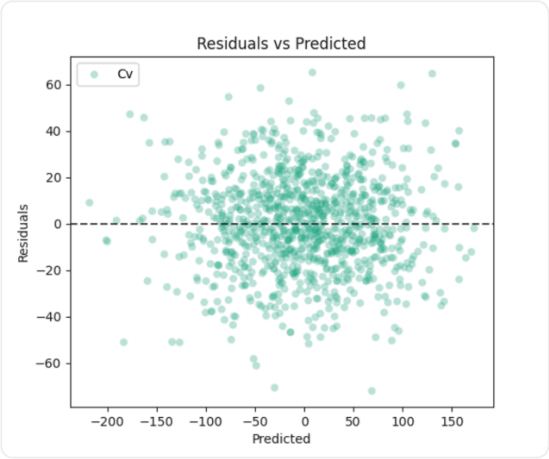
Max Error	66.7035	4.0403	61.1649	72.0428
Median Absolute Error	13.2152	0.9034	12.2722	14.8012
Mean Squared Error	430.8122	25.4447	386.9170	463.2783
R^2 Score	0.9045	0.0053	0.8942	0.9091
Root Mean Squared Error	20.7468	0.6190	19.6702	21.5239

VISUALIZATIONS









5. Grid Search Example (Classification)

Classification Report (Hyperparameter Search)

Lucas Summers
KNN tuning demo

MODEL OVERVIEW

Name	KNeighborsClassifier
Type	Classification
Sklearn	1.8.0

HYPERPARAMETERS

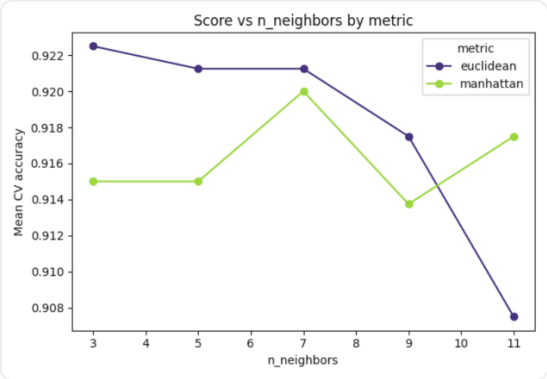
algorithm	auto	leaf_size	30
metric	**euclidean**	metric_params	None
n_jobs	None	**n_neighbors**	**3**
p	2	weights	uniform

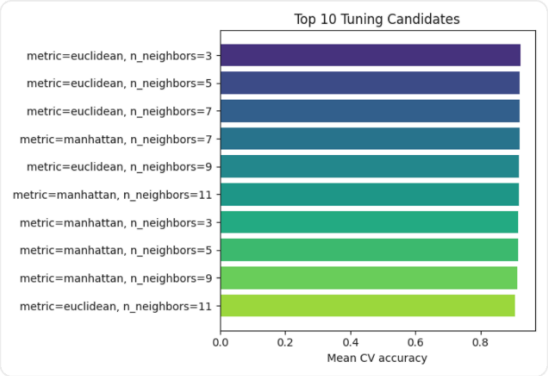
Tuning Summary

Method	GridSearchCV
Metric	accuracy
CV Folds	5
Candidates	10
Best Score	0.9225

5. Grid Search Example (Classification)

Tuning Plots





DATASET OVERVIEW

Features	10
Total Observations	1000
CV Folds	None
Train	800 (80.0%)
Test	200 (20.0%)

Class Distribution

Class	Train	Test	Overall %
0	398	100	49.8%
1	402	100	50.2%

METRICS (TRAIN/TEST SPLIT)

Metric	Train	Test	Gap
Accuracy	0.9587	0.9300	+0.0287
F1 Score (Macro)	0.9587	0.9300	+0.0287

5. Grid Search Example (Classification)

F1 Score (Weighted)	0.9587	0.9300	+0.0287
Precision (Macro)	0.9588	0.9300	+0.0288
Precision (Weighted)	0.9588	0.9300	+0.0288
Recall (Macro)	0.9587	0.9300	+0.0287
Recall (Weighted)	0.9587	0.9300	+0.0287

*Gap = Train - Test. Large positive values may indicate overfitting.

METRICS (PER-CLASS)

Train

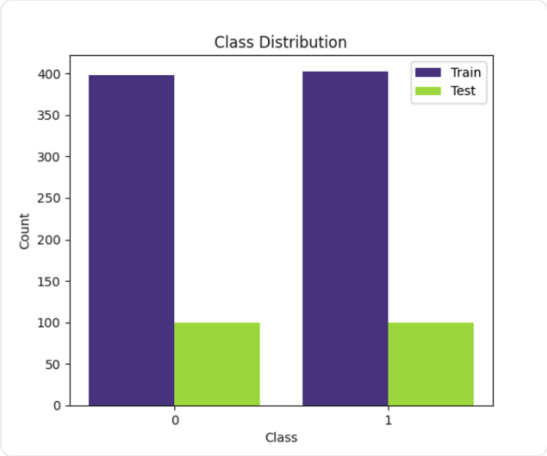
Class	Precision	Recall	F1
0	0.9620	0.9548	0.9584
1	0.9556	0.9627	0.9591

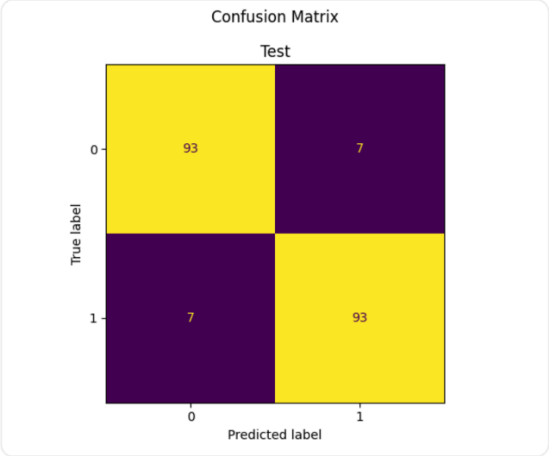
Test

Class	Precision	Recall	F1
0	0.9300	0.9300	0.9300
1	0.9300	0.9300	0.9300

5. Grid Search Example (Classification)

VISUALIZATIONS





6. Grid Search Example (Regression)

Regression Report (Categorical x Categorical Search)

Lucas Summers
CSC 466-02

MODEL OVERVIEW

Name	RandomForestRegressor
Type	Regression
Sklearn	1.8.0

HYPERPARAMETERS

bootstrap	**True**	ccp_alpha	0.0
criterion	**squared_error**	max_depth	None
max_features	1.0	max_leaf_nodes	None
max_samples	None	min_impurity_decrease	0.0
min_samples_leaf	1	min_samples_split	2
min_weight_fraction_leaf	0.0	monotonic_cst	None
n_estimators	100	n_jobs	None
oob_score	False	random_state	42
verbose	0	warm_start	False

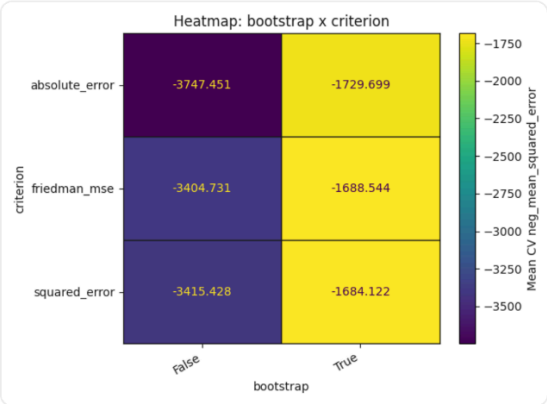
Tuning Summary

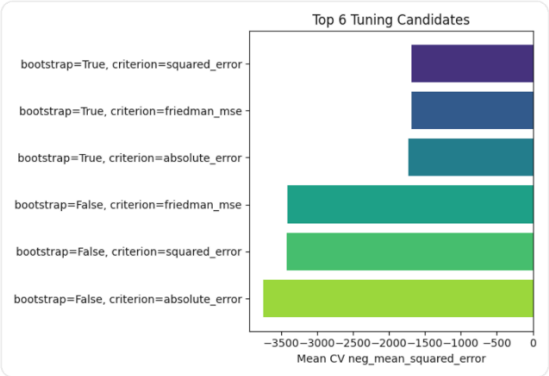
Method	GridSearchCV
Metric	neg_mean_squared_error

6. Grid Search Example (Regression)

CV Folds	5
Candidates	6
Best Score	-1684.1223

Tuning Plots





DATASET OVERVIEW

Features	10
Total Observations	1000
CV Folds	None
Train	800 (80.0%)
Test	200 (20.0%)

METRICS (TRAIN/TEST SPLIT)

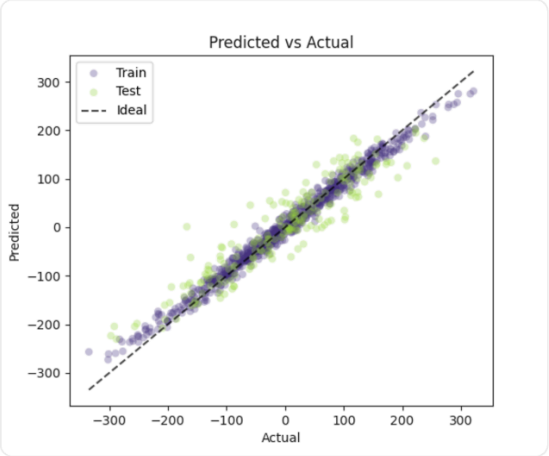
Metric	Train	Test	Gap
Mean Absolute Error	11.5573	31.9806	-20.4234
Max Error	78.1765	168.8035	-90.6270

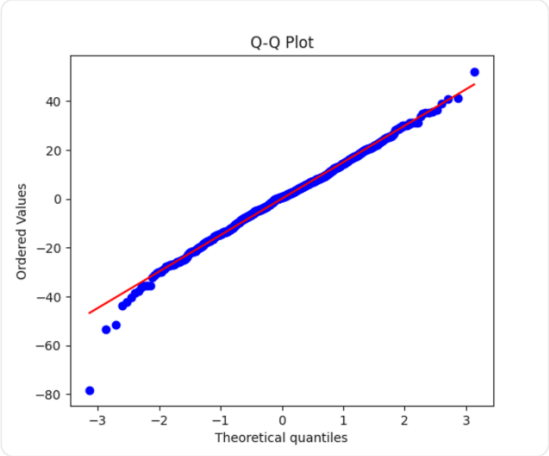
6. Grid Search Example (Regression)

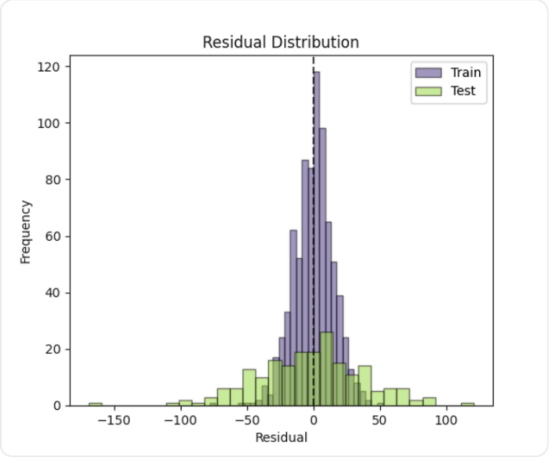
Median Absolute Error	9.1201	25.5297	-16.4095
Mean Squared Error	223.6798	1721.9284	-1498.2487
R^2 Score	0.9813	0.8632	+0.1181
Root Mean Squared Error	14.9559	41.4961	-26.5402

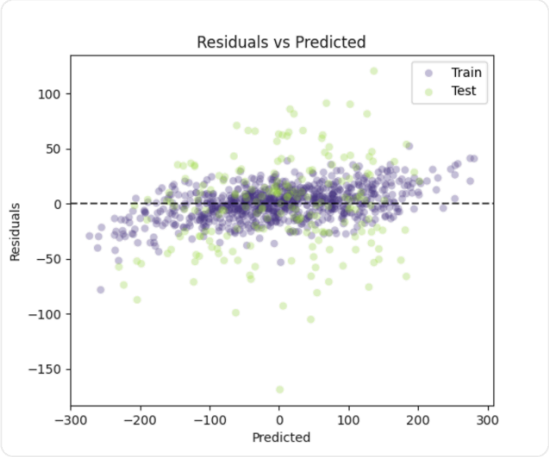
*Gap = Train - Test. Large positive values may indicate overfitting.

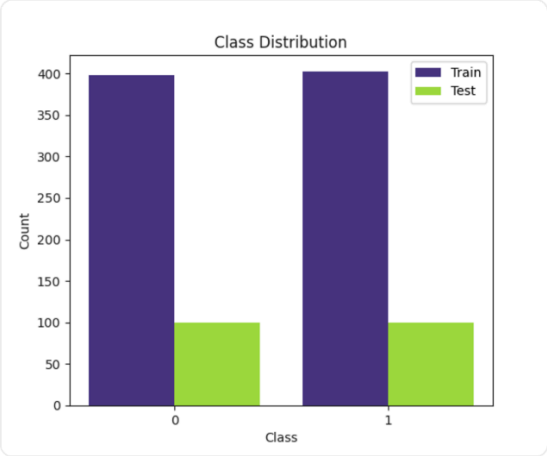
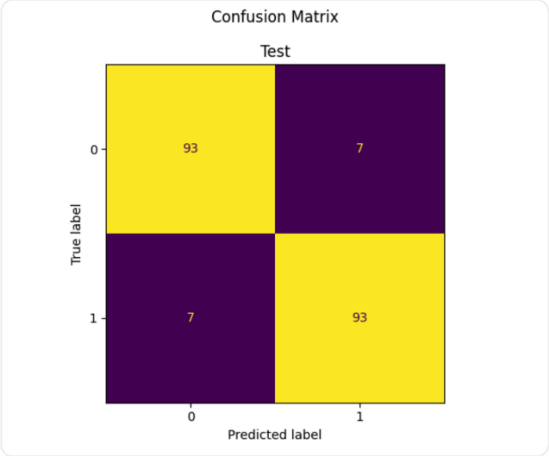
VISUALIZATIONS









6. Grid Search Example (Regression)

VISUALIZATIONS